



Die Spiele-Schmiede: Vom Anfänger zum Spiele-Programmierer (Zero to Hero)

Herzlich willkommen in deiner eigenen Spiele-Werkstatt! In diesem Handbuch lernst du Schritt für Schritt, wie du mit echtem C++ eigene Retro-Spiele (wie Pong, Snake, Breakout oder Space-Shooter) programmierst.



Kapitel 0: Das große Geheimnis der Videospiele

Hast du schon mal ein **Daumenkino** gebastelt? Wenn man schnell durch die Seiten blättert, erwachen die gezeichneten Figuren zum Leben!

Genauso funktioniert dein Computer:

- Unser Spiel malt **60-mal in jeder einzelnen Sekunde** ein neues Bild auf den Bildschirm.
- Das Herzstück jedes Spiels ist diese eine Funktion:

```
void update(Engine& e) {  
    // Alles hier drin wird 60-mal pro Sekunde ausgeführt!  
}
```

(Hinweis: Warum hinter `Engine` ein `&`-Zeichen steht, erfährst du im [Anhang B](#) – am Anfang musst du dir darüber keine Sorgen machen!)



Das C++ Zauberbuch: Die wichtigsten Grundlagen

C++ ist eine der schnellsten und mächtigsten Programmiersprachen der Welt. Fast alle großen Konsolen- und PC-Spiele werden in C++ geschrieben. Hier sind die wichtigsten Bausteine, die du brauchst:

1. Variablen (Die beschrifteten Schachteln)

In Variablen merkt sich der Computer Zahlen, Texte und Zustände:

- `int` (Ganze Zahlen): Für Punktestände, Leben oder Zähler.

```
int score = 0;  
int leben = 3;
```

- **float** (Kommazahlen): Für Positionen und Geschwindigkeiten (damit Bewegungen butterweich sind).

```
float ballX = 160.0f;
float geschwindigkeit = 3.5f;
```

- **bool** (Schalter: **true** / **false**): Für Ja/Nein-Zustände.

```
bool spielAktiv = true;
bool laserBereit = false;
```

2. Entscheidungen treffen (**if** und **else**)

Mit **if** („Wenn...“) kann dein Spiel auf Ereignisse reagieren:

```
if (ballX > 320) {
    // Tor erzielt!
    score = score + 1;
    e.play_tone(Notes::C5);
} else {
    // Sonst: Ball ist noch im Spielfeld
}
```

Die Vergleichs-Zeichen:

- **==** Ist gleich? (**if (leben == 0)**)
- **!=** Ist ungleich? (**if (richtung != Direction::Down)**)
- **<**, **>**, **<=**, **>=** Kleiner, Größer, Kleiner-Gleich, Größer-Gleich
- **&&** **UND** (**if (ballX > 0 && ballX < 320)**)
- **||** **ODER** (**if (e.key(Key::W) || e.key(Key::Up))**)

3. Wiederholungen (**for** -Schleifen)

Wenn du viele Dinge auf einmal tun willst (z. B. 70 Sterne zeichnen oder 5 Äpfel platzieren), benutzt du eine **for** -Schleife:

```
// Zählt von 0 bis 4 (wiederholt sich 5-mal):
for (int i = 0; i < 5; ++i) {
    spawnApple();
}
```

4. Baupläne für Spiele & Objekte (struct)

Ein `struct` ist ein **Bauplan** für ein ganzes Spiel oder ein Objekt (wie einen Stern oder Laser). In jedem Spiel ordnen wir den Code von oben nach unten in 4 klare Abschnitte:

```
struct MeinSpiel : Game {
    // 1. VARIABLEN (Zustand & Eigenschaften ganz oben!)
    float ballX = 160.0f;
    float ballY = 120.0f;
    int score = 0;

    // 2. KONSTRUKTOR (Startklar machen!)
    // Heißt wie das struct und wird beim Start 1-mal ausgeführt:
    MeinSpiel() {
        score = 0;
    }

    // 3. HAUPTSCHLEIFE (60-mal pro Sekunde)
    void update(Engine& e) override {
        e.cls(Colors::DarkGreen);
        bewegeBall(); // Ruft unsere Methode auf
        zeichneBall(e);
    }

    // 4. METHODEN (Eigene Hilfs-Aktionen)
    void bewegeBall() {
        // DER ZAUBERTRICK: Methoden können direkt auf ALLE
        // Variablen oben (ballX, ballY, score) zugreifen!
        ballX += 2.0f;
    }

    void zeichneBall(Engine& e) {
        e.circlefill(ballX, ballY, 8, Colors::Yellow);
    }
};
```

💡 Was ist der Unterschied zwischen Funktion und Methode?

Eine **Funktion**, die innerhalb* eines `struct` s steht, nennt man **Methode**.

* **Der Super-Vorteil:** Alle Methoden im selben `struct` haben wie durch Gedankenübertragung **automatisch Zugriff auf alle Variablen** ganz oben! Du musst `ballX` oder `score` nicht extra übergeben.

💡 Der Geheimtipp: `struct` vs. `class` :

In C++ sind `struct` und `class` fast genau dasselbe! Der einzige Unterschied: Bei `struct` ist standardmäßig alles öffentlich zugänglich. Bei `class` müsste man überall extra `public:` davorschreiben. Mit `struct` sparen wir uns lästiges Tippen!

5. Die `std` Standard-Bibliothek (Schlaue Werkzeuge)

C++ bringt einen Werkzeugkasten namens `std::` (Standard Library) mit:

- `std::vector` (Die flexible Liste):

Ein Vector ist wie ein Gummiband-Array. Er wächst automatisch, wenn neue Dinge dazukommen:

```
std::vector<Star> stars;    // Liste aller Sterne
stars.push_back(neuerStern); // Stern hinten anfügen
stars.pop_back();          // Stern entfernen
int anzahl = stars.size();  // Wie viele Sterne?
```

- `std::deque` (Die Schlange / Kette):

Perfekt für Snake! Bei einer Deque kannst du blitzschnell vorne anbauen und hinten abschneiden:

```
body.push_front(neuerKopf); // Vorne Kopf anfügen
body.pop_back();           // Hinten Schwanz kappen
```

- `std::clamp` (Der Begrenzer):

Hält einen Wert sauber zwischen Minimum und Maximum (damit Schläger nicht aus dem Bildschirm fahren):

```
playerY = std::clamp(playerY, 18.0f, 200.0f);
```

6. Echtes Hacken: Dein Spiel im Terminal kompilieren!

Ein **Compiler** übersetzt deinen C++ Code in Maschinensprache, die der Prozessor direkt ausführt.

Du kannst dein Spiel direkt im Terminal (Shell) von Hand kompilieren:

```
# 1. Den C++ Compiler 'g++' aufrufen:  
g++ -std=c++17 -I/opt/homebrew/include main.cc \  
-o mein_spiel -L/opt/homebrew/lib \  
-lsfml-graphics -lsfml-window \  
-lsfml-system -lsfml-audio  
  
# 2. Dein fertiges Spiel starten:  
./mein_spiel
```

Oder noch einfacher mit unserem **Makefile**:

```
make          # Kompiliert das Spiel  
./test        # Startet das Spiel  
make static   # Baut Datei ohne externe Abhängigkeiten  
make bundle   # Baut echte macOS App: MTG.app!
```

Level 1: Die Malerwerkstatt (Farben & Formen)

1. Die Zauberleinwand & Koordinaten

Unser Bildschirm ist **320 Pixel breit** und **240 Pixel hoch**.

- **X (nach rechts)**: Geht von 0 (ganz links) bis 320 (ganz rechts).
- **Y (nach unten)**: Geht von 0 (ganz oben) bis 240 (ganz unten).
- Die Ecke oben links ist **(0, 0)**!

2. Der Farbkasten

Du kannst diese Farben überall verwenden:

- `Colors::DarkGreen` (Dunkelgrüner Rasen)
- `Colors::Red` (Leuchtendes Rot)
- `Colors::Blue` (Helles Blau)
- `Colors::Yellow` (Sonnengelb)
- `Colors::White` (Schneeweiß)
- `Colors::Black` (Tiefschwarz)
- `Colors::Orange`, `Colors::LightGreen`, `Colors::DarkGray`, `Colors::LightGray`

3. Deine Mal-Werkzeuge

Befehl	Was er macht	Beispiel
<code>e.cls(Farbe)</code>	Tafel putzen: malt den ganzen Bildschirm an.	<code>e.cls(Colors::DarkGreen);</code>
<code>e.pset(x, y, Farbe)</code>	Punkt malen: Setzt einen einzelnen Pixel.	<code>e.pset(160, 120, Colors::Yellow);</code>
<code>e.line(x1, y1, x2, y2, Farbe)</code>	Linie ziehen: Verbindet zwei Punkte.	<code>e.line(0, 0, 320, 240, Colors::White);</code>
<code>e.rect(x, y, b, h, Farbe)</code>	Rechteck-Rahmen: Zeichnet Kasten-Umriss.	<code>e.rect(10, 10, 50, 30, Colors::Red);</code>
<code>e.rectfill(x, y, b, h, Farbe)</code>	Rechteck füllen: Ausgefüllter Kasten.	<code>e.rectfill(10, 10, 50, 30, Colors::Red);</code>
<code>e.circle(x, y, r, Farbe)</code>	Kreis-Rahmen: Zeichnet Kreis-Umriss.	<code>e.circle(160, 120, 20, Colors::White);</code>
<code>e.circlefill(x, y, r, Farbe)</code>	Ball malen: Ausgefüllter Kreis.	<code>e.circlefill(160, 120, 8, Colors::Yellow);</code>
<code>e.draw_text(x, y, "TEXT", Farbe)</code>	Schrift schreiben: Text anzeigen.	<code>e.draw_text(100, 50, "HALLO", Colors::White);</code>
<code>e.draw_digit(x, y, zahl, Farbe)</code>	Große Zahl: Punktestände.	<code>e.draw_digit(100, 20, 5, Colors::Yellow, 3);</code>

🎯 **Mini-Quest 1:** Male ein Haus mit einem dreieckigen Dach (aus Linien) und einer bunten Tür!



Level 2: Leben einhauchen! (Variablen & Bewegung)

Was ist eine Variable?

Eine Variable ist wie eine **beschriftete Schachtel**, in der sich eine Zahl merkt:

```
// Eine Schachtel namens ballX mit dem Wert 100:
float ballX = 100.0f;
```

Die Zauberformel für Bewegung

Wenn wir in jedem Bild (`update`) die Zahl ein kleines Stückchen verändern, bewegt sich das Objekt:

```
struct MeinSpiel : Game {
    float ballX = 0.0f;

    void update(Engine& e) override {
        e.cls(Colors::DarkGreen);

        // Den Ball ein Stück nach rechts schieben:
        ballX = ballX + 2.0f;

        // Ball an der neuen Position malen:
        e.circlefill(ballX, 120, 8, Colors::Yellow);
    }
};
```

Level 3: Die Fernbedienung (Tastatur & Maus)

Jetzt machen wir das Spiel interaktiv!

Tasten abfragen:

- `e.key(Key::...)` → Prüft, ob eine Taste **gedrückt gehalten** wird (für Schläger / Raumschiffe):

```
if (e.key(Key::W)) {
    spielerY = spielerY - 4.0f; // Nach oben
}
if (e.key(Key::S)) {
    spielerY = spielerY + 4.0f; // Nach unten
}
```

- `e.pressed(Key::...)` → Prüft, ob eine Taste **gerade angetippt** wurde (für Schuss oder Sprung):

```
if (e.pressed(Key::Space)) {
    e.play_tone(Notes::As5, 0.1f); // Schuss-Ton!
}
```

Verfügbare Tasten:

- `Key::W`, `Key::S`, `Key::A`, `Key::D`

- `Key::Up`, `Key::Down`, `Key::Left`, `Key::Right`
- `Key::Space`, `Key::Enter`, `Key::Escape`
- `Key::Num1`, `Key::Num2`, `Key::Num3`

Mit der Maus malen:

- `e.mouse_x()` und `e.mouse_y()` verraten dir die Position des Mauszeigers.
- `e.mouse_down()` prüft, ob die Maustaste gedrückt ist:

```
if (e.mouse_down()) {
    e.circlefill(e.mouse_x(), e.mouse_y(), 5, Colors::Yellow);
}
```

🔥 Level 4: Kollisionen & Spielphysik

1. Wand-Abprall (Der Flummi-Trick)

Wenn der Ball oben oder unten anstößt, kehren wir einfach seine vertikale Geschwindigkeit (`ballVy`) um:

```
ballY = ballY + ballVy;

if (ballY <= 10 || ballY >= 230) {
    // Richtung umdrehen: Aus + wird - und umgekehrt!
    ballVy = -ballVy;
    e.play_tone(Notes::G4, 0.05f); // Abprall-Ton!
}
```

2. Schläger trifft Ball (Kollisionsprüfung)

Wir prüfen, ob sich der Ball innerhalb der Schläger-Grenzen befindet:

```
if (ballX <= schlaegerX + schlaegerBreite &&
    ballY >= schlaegerY &&
    ballY <= schlaegerY + schlaegerHoehe) {
    // Ball fliegt wieder nach rechts zurück:
    ballVx = -ballVx;
    e.play_tone(Notes::G5, 0.08f); // Treffer-Ton!
}
```

🎵 Level 5: Töne & Musik (Der Noten-Baukasten)

Töne machen ein Spiel erst richtig lebendig! Du musst dir keine komplizierten Hertz-Zahlen (wie 311.13 Hz) merken, sondern kannst einfach echte **Notennamen** (`Notes::...`) verwenden:

1. Die Notennamen (`Notes::...`)

- **Oktave 3 (Bass):** `Notes::C3`, `Notes::E3`, `Notes::G3`, `Notes::A3`, etc.
- **Oktave 4 (Klavier-Mitte):** `Notes::C4` (Mittleres C), `Notes::D4`, `Notes::E4`, `Notes::F4`, `Notes::G4`, `Notes::A4` (440 Hz), `Notes::B4`
- **Oktave 5 (Hohe Melodie):** `Notes::C5`, `Notes::D5`, `Notes::E5`, `Notes::F5`, `Notes::G5`, `Notes::A5`, `Notes::B5`
- **Oktave 6 (Piepser/Glocken):** `Notes::C6`, `Notes::E6`, `Notes::G6`
- *Tipp für Halbtöne:* Ein `s` steht für Kreuz (#) wie `Notes::Fs4` (Fis) oder `Notes::Cs5` (Cis).

2. Einzelne Töne abspielen:

`e.play_tone(Note, DauerInSekunden)` :

- **Heller Piep-Ton (z. B. Sprung / Treffer):**

```
e.play_tone(Notes::G5, 0.08f);
```

- **Tiefer Wumm (z. B. Wandstoß / Explosion):**

```
e.play_tone(Notes::C3, 0.20f);
```

- **Sehr hoher Münzen-Sound:**

```
e.play_tone(Notes::E6, 0.06f);
```

3. Eigene Melodien komponieren:

Mit `e.play_melody` kannst du mehrere Noten nacheinander abspielen:

```
// Eine fröhliche 3-Ton-Fanfare:
e.play_melody({
    {Notes::C5, 0.10f}, // Ton C
    {Notes::E5, 0.10f}, // Ton E
    {Notes::G5, 0.25f}  // Ton G (länger)
});

// Eine traurige Verlierer-Melodie:
e.play_melody({
    {Notes::A4, 0.15f},
    {Notes::Fs4, 0.15f},
    {Notes::Ds4, 0.18f},
    {Notes::A3, 0.40f}
});
```

Level 6: Spiel 1 – Das Retro-Pong Match (PongGame.hpp)

In **Pong** lernen wir, wie zwei Spieler gegeneinander antreten:

1. **Zwei Schläger:** Spieler 1 (Rot mit `W` / `S`), Spieler 2 (Blau mit Pfeiltasten).
2. **Dynamischer Abprallwinkel:** Trifft der Ball die Schlägermitte, fliegt er geradeaus. Trifft er den Rand, prallt er im steilen Winkel ab!
3. **Tempo-Steigerung:** Bei jedem Schlag wird der Ball 5% schneller (`speed *= 1.05f`).
4. **Spielphasen mit `enum class` & `switch`**: Wie das Spiel zwischen Startmenü, Ballwechsel und Sieg umschaltet (*Details siehe [Anhang A](#)*).

Level 7: Spiel 2 – Das Schlangen-Duell (SnakeGame.hpp)

In **Snake** lernen wir zwei mächtige neue Programmier-Tricks kennen:

1. Das Kachel-Gitter (Grid)

Statt in einzelnen Pixeln rechnen wir hier in Kacheln (jede Kachel ist 8x8 Pixel groß):

- Unser Spielfeld hat **40 Kacheln Breite** und **30 Kacheln Höhe**.
- Eine Kachel-Position ist einfach:

```
struct Cell { int x, y; };
```

2. Wie bewegt sich eine Schlange?

Die Schlange ist eine Kette von Kacheln (`std::deque<Cell>`). In jedem Schritt passiert Magie:

```
// 1. Vorne einen neuen Kopf in Gehrichtung anfügen:
body.push_front(neuerKopf);

// 2. Hat die Schlange einen Apfel gefressen?
if (apfelGefressen) {
    // Schlange wächst (Schwanz bleibt erhalten):
    score++;
} else {
    // Schwanz nachziehen (letztes Glied kappen):
    body.pop_back();
}
```

3. Der „Apfel-Klau“-Mechanismus (Modell B)

Wenn eine Schlange an die Wand oder in den Gegner fährt:

```
// Jeden Körperteil in Apfel auf dem Feld verwandeln:
for (size_t i = 1; i < body.size(); ++i) {
    apples.push_back(body[i]);
}
// Schlange startet sofort wieder klein in ihrer Ecke neu:
respawnSnake();
```

Der überlebende Spieler kann die Trümmer-Äpfel blitzschnell fressen und riesig werden!

Level 8: Spiel 3 – Die Weltraum-Schlacht (StarfighterGame.hpp)

Hier steigen wir ein in die Welt der **Partikel-Systeme** und **Pixel-Art-Schiffe**:

1. Das 3D-Parallaxe Sternenfeld

Um dem Weltraum echte Tiefe zu verleihen, lassen wir 70 Sterne in **3 verschiedenen Tiefen-Ebenen** fliegen:

- **Hintergrund** (dunkelgrau, langsam: 25 px/s) → Weit entfernte Sterne
- **Mittlere Sterne** (hellgrau, mittel: 65 px/s)
- **Vordergrund** (strahlend weiß, schnell: 130 px/s) → Nahe Sterne

```

struct Star {
    float x, y, speed;
    uint8_t color;
};

// In jedem Frame nach unten wandern:
// (Warum hier e.dt() steht, erfährst du im Anhang D!)
s.y += s.speed * e.dt();
if (s.y >= 240.0f) {
    s.y = 0.0f;
    s.x = rand() % 320;
}

```

2. 16x16 Pixel-Art Raumschiffe mit Bitmasken

Wir können Raumschiffe direkt im Code als Pixelmuster zeichnen:

```

// Das 16x16 Delta X-Wing Schiff (Zeile für Zeile):
static const uint16_t playerSprite[16] = {
    0b0000000011000000, // Spitze
    0b0000000111100000, // Cockpit
    0b0100001111000010, // Flügelkanonen vorn
    0b0100111111110010, // Delta-Flügel
    0b1111111111111111, // Rumpf
    0b0000000100100000, // Schubflamme
};

```

3. Schiffs-Explosionen mit echter Physik (Trägheit & Farbverlauf)

Wenn ein Schiff explodiert, sprengen sich die Trümmer nicht nur im Kreis auseinander, sondern **nehmen den vollen Geschwindigkeitsvektor des Schiffs mit**:

```

// 1. Schiffsimpuls + radiale Sprengung (Sinus/Kosinus aus Anhang E):
p.vx = shipVx + burstSpeed * cos(angle);
p.vy = shipVy + burstSpeed * sin(angle);

// 2. Thermischer Farbverlauf über Lebensdauer:
// Weiß -> Gelb -> Orange -> Rot -> Dunkelgrau

```



Level 9: Die Retro-Konsole (MenuGame.hpp)

Wie baut man ein Hauptmenü, um zwischen mehreren Spielen umzuschalten?

```
struct MenuGame : Game {
    std::unique_ptr<PongGame> pong;
    std::unique_ptr<SnakeGame> snake;
    std::unique_ptr<StarfighterGame> starfighter;

    void update(Engine& e) override {
        // Mit Escape immer zurück ins Menü:
        if (e.pressed(Key::Escape)) {
            currentGame = CurrentGame::Menu;
        }

        if (e.pressed(Key::Num1)) { currentGame = CurrentGame::Pong; }
        if (e.pressed(Key::Num2)) { currentGame = CurrentGame::Snake; }
        if (e.pressed(Key::Num3)) { currentGame = CurrentGame::Starfighter; }
    }
};
```

Der Spickzettel (Cheat Sheet) auf einen Blick

```
// --- Malen ---
e.cls(Farbe); // Bildschirm leeren
e.pset(x, y, Farbe); // Pixel setzen
e.line(x0, y0, x1, y1, Farbe); // Linie ziehen
e.rectfill(x, y, breite, hoehe, Farbe); // Ausgefülltes Rechteck
e.circlefill(x, y, radius, Farbe); // Ausgefüllter Kreis
e.draw_text(x, y, "TEXT", Farbe); // Text schreiben
e.draw_digit(x, y, zahl, Farbe, 3); // Große Ziffer malen

// --- Steuerung ---
if (e.key(Key::W)) { ... } // Taste gedrückt halten
if (e.pressed(Key::Space)) { ... } // Taste neu angetippt
e.mouse_x(); // Maus X
e.mouse_y(); // Maus Y
e.mouse_down(); // Maustaste gedrückt?

// --- Sound & Noten ---
e.play_tone(Notes::C5, 0.1f); // Einzelnen Ton abspielen
e.play_melody({ // Melodie abspielen
    {Notes::C5, 0.1f},
    {Notes::E5, 0.1f},
    {Notes::G5, 0.2f}
});

// --- Farben ---
Colors::DarkGreen, Colors::Red, Colors::Blue,
Colors::Yellow, Colors::White, Colors::Black,
Colors::Orange, Colors::LightGreen, Colors::DarkGray, Colors::LightGray
```

Anhang: Für Fortgeschrittene & Neugierige

 **Hinweis:** Diesen Anhang musst du am Anfang noch nicht auswendig wissen! Schau einfach hier rein, wenn du wissen willst, was bestimmte Profi-Zeichen (wie `&` oder `const`) bedeuten oder wie besondere Programmiertricks im Detail funktionieren.

Anhang A: `enum class` & `switch-case` (Die Spielphasen-Weiche)

1. Was ist ein `enum class`?

Statt dir für Spielphasen kryptische Zahlen zu merken (`0 = Start`, `1 = Spielt`, `2 = Vorbei`), erfindest du mit `enum class` deine eigenen, sprechenden Wörter:

```
enum class State {
    Title,          // Startbildschirm
    Playing,        // Spiel läuft
    PointScored,    // Tor erzielt
    Victory         // Jemand hat gewonnen
};
```

```
State state = State::Title;
```

2. Was ist `switch` und `case` ?

Ein `switch` ist wie eine **Weiche auf einer Eisenbahnschiene**. Statt viele `if ... else if ... else if` zu schreiben, springt das Programm direkt zur passenden Station:

```
switch (state) {
case State::Title:
    updateTitle(e);
    break; // Fertig, Weiche verlassen!
case State::Playing:
    updatePlaying(e);
    break;
case State::Victory:
    updateVictory(e);
    break;
}
```

Anhang B: Das Geheimnis des `&`-Zeichens (Referenzen)

Vielleicht ist dir aufgefallen, dass bei `void update(Engine &e)` ein kleines `&` hinter `Engine` steht.

Was bedeutet das?

- **Ohne `&`**: Der Computer würde eine komplette **Kopie** der Spiele-Engine anfertigen. Das ist wie das Foto einer Fernbedienung – wenn du auf das Foto drückst, passiert am Fernseher gar nichts!
- **Mit `&` (Referenz)**: Wir übergeben dem Spiel das **echte Original**. Wenn das Spiel `e.cls()` ruft, malt es direkt auf den echten Monitor. Außerdem ist es viel schneller, weil der Computer nichts kopieren muss.

Anhang C: `const` & `constexpr` (Der Schreibschutz)

Wenn vor einer Variable `const` steht:

```
const float maxBallSpeed = 320.0f;
```

bedeutet das: **Dieser Wert ist unveränderlich (Konstante)!**

- Es ist wie ein **Sicherheitsgurt**: Wenn du später im Code aus Versehen `maxBallSpeed = 0` schreiben würdest, warnt dich der Compiler sofort mit einem roten Fehler.
- `constexpr` bedeutet, dass der Wert schon beim Kompilieren feststeht (z. B. für Kachelgrößen wie `static constexpr int CELL_SIZE = 8;`).



Anhang D: Delta-Time (`e.dt()`) – Warum das Spiel immer gleich schnell läuft

Hast du einen Monitor mit 60 Hz, 120 Hz oder 240 Hz?

- Wenn wir einfach `ballX += 3.0f` pro Bild rechnen, würde der Ball auf einem 120 Hz Monitor **doppelt so schnell fliegen** wie auf einem 60 Hz Monitor!
- Die Lösung heißt **Delta-Time (`e.dt()`)**: Das ist die exakte Zeit in Sekunden, die seit dem letzten Bild vergangen ist (z. B. `0.0166` Sekunden bei 60 FPS).
- Wenn wir rechnen:

```
ballX += 180.0f * e.dt(); // 180 Pixel pro Sekunde
```

bewegt sich der Ball auf jedem Computer der Welt **haargenau gleich schnell!**



Anhang E: Mathe-Zauber mit `<cmath>` (Sinus, Kosinus & Abstand)

In `<cmath>` stecken clevere mathematische Werkzeuge:

1. Sinus & Kosinus (`std::sin` , `std::cos`)

- **Kreisbewegungen & Explosionen**: Mit `cos(winkel)` und `sin(winkel)` kannst du Trümmer perfekt in alle 360 Grad im Kreis wegschleudern!
- **Schlangenlinien**: Mit `y = startY + sin(zeit 3.0f) 20.0f` fliegt ein Raumschiff in einer sanften Wellenlinie über den Bildschirm.

2. Der Betrag / Abstand (`std::abs`)

- `std::abs(zahl)` macht aus jeder negativen Zahl eine positive (`std::abs(-5) = 5`).
- Perfekt für Kollisionen: `std::abs(playerX - enemyX) < 14.0f` prüft blitzschnell, ob zwei Schiffe näher als 14 Pixel beieinander sind!

3. Zufallszahlen (`rand()` % N)

- `rand() % 320` würfelt eine Zufallszahl zwischen 0 und 319 (perfekt, um Sterne oder Äpfel zufällig zu verteilen).

Anhang F: Header-Dateien & `#pragma once`

Warum haben unsere Dateien die Endung `.hpp`?

- `.hpp` steht für **C++ Header**.
- Jedes Spiel (`PongGame.hpp`, `SnakeGame.hpp`, `StarfighterGame.hpp`) wohnt in seiner eigenen Datei.
- In `main.cc` binden wir sie mit `#include "PongGame.hpp"` einfach ein wie Bausteine.
- `#pragma once` ganz oben in der Datei sorgt dafür, dass der Computer die Datei niemals versehentlich doppelt liest.

Anhang G: Coole Ideen für deine nächsten Spiele!

Hier sind ein paar spannende Herausforderungen, wenn du Lust hast, eigene Spiele zu erfinden:

1. **Computer-Gegner für Pong:**

Lass den rechten Schläger automatisch der Y-Position des Balls folgen:

```
if (p2Y + 18.0f < ballY) { p2Y += paddleSpeed * e.dt(); }
if (p2Y + 18.0f > ballY) { p2Y -= paddleSpeed * e.dt(); }
```

1. **Breakout / Stein-Zerstörer:**

Platziere oben ein Gitter aus bunten Steinen (`struct Brick { float x, y; bool alive; };`).

Trifft der Ball einen Stein, verschwindet er und gibt Punkte!

1. **Schild / Lebensbalken für Starfighter:**

Lass dein Raumschiff 3 Treffer aushalten und zeichne oben einen grünen Energiebalken mit

```
e.rectfill(20, 10, lebenspunkte * 20, 6, Colors::LightGreen);!
```

1. **Highscore auf der Festplatte speichern:**

Mit C++ Dateibefehlen (`std::ofstream` / `std::ifstream`) kannst du den Rekord in eine Textdatei schreiben, damit er nach dem Ausschalten nicht verloren geht!

Viel Spaß beim Erfinden, Experimentieren und Programmieren!  